

C++ 연습 문제: 03. 클래스와 객체

1. 다음 코드에서 외부 코드의 직접 접근을 막고 멤버 변수를 안전하게 숨기기 위해 빈칸 (A)에 들어갈 접근 제어 지시자는 무엇인가? (하)

```
class Account {  
    ( A ):  
        int balance;  
public:  
        void Deposit(int amount) { balance += amount; }  
};
```

1. public
2. private
3. protected
4. friend

2. C++에서 class와 struct에 대한 설명으로 올바른 것은? (하)

1. struct는 멤버 함수를 포함할 수 없고 오직 데이터 변수만 선언할 수 있다.
2. 접근 제어 지시자를 생략했을 때 class는 private, struct는 public이 기본값으로 적용된다.
3. class와 struct는 상속 및 다형성 기능 지원 여부에서 본질적인 차이가 있다.
4. struct로 선언된 객체는 stack이 아닌 heap에만 할당된다.

3. 다음 코드처럼 클래스 외부에 멤버 함수를 정의할 때 소속 클래스를 지정하기 위해 빈칸 (A)에 들어갈 기호는 무엇인가? (하)

```
class Character {  
public:  
        void Move(int x, int y);  
};  
  
void Character( A )Move(int x, int y) {  
    // 이동 로직 구현  
}
```

1. .
2. ->
3. ::
4. :

4. 멤버 변수 이름과 매개변수 이름이 같을 때, 매개변수가 아닌 멤버 변수를 명확히 가리키기 위해 빈칸 (A)에 들어갈 키워드는 무엇인가? (중)

```
class Vector2D {
    float x, y;
public:
    void SetX(float x) {
        ( A )->x = x;
    }
};
```

1. this
2. self
3. that
4. base

5. 다음 코드는 멤버 함수를 연속적으로 호출하는 메서드 체이닝(Method Chaining)을 구현한다. 빈칸 (A)에 들어갈 올바른 반환 코드는 무엇인가? (상)

```
class Monster {
    int hp = 100;
public:
    Monster& SetHp(int h) {
        hp = h;
        return ( A );
    }
};
```

1. this
2. *this
3. &this
4. self

6. 정적(static) 멤버 함수에 대한 설명으로 올바른 것은? (중)

1. 객체를 생성하지 않고도 클래스명::함수명() 형태로 호출할 수 있으며 this 포인터를 가지지 않는다.
2. 정적 멤버 함수 내부에서는 비정적 인스턴스 멤버 변수에 직접 접근할 수 있다.
3. 정적 멤버 함수는 항상 const 키워드를 붙여 선언해야 한다.
4. 정적 멤버 함수는 가상 함수(virtual function)로 선언하여 오버라이딩할 수 있다.

7. 다음 코드에서 클래스 내부에 선언된 정적 데이터 멤버 itemCount를 클래스 외부에서 올바르게 정의 및 초기화하는 빈칸 (A)의 문법은 무엇인가? (중)

C++

```
class Item {
public:
    static int itemCount;
};
```

```
// 클래스 외부 정의
( A ) = 0;
```

1. static int Item::itemCount
2. int Item::itemCount
3. int Item.itemCount
4. Item::itemCount

8. 다음 코드에서 전역 함수 ShowSecret이 Vault 클래스의 private 멤버에 접근할 수 있도록 권한을 부여하는 빈칸 (A)의 키워드는 무엇인가? (중)

```
class Vault {
private:
    int secretCode = 1234;
    ( A ) void ShowSecret(const Vault& v);
};

void ShowSecret(const Vault& v) {
    std::cout << v.secretCode;
}
```

1. public
2. inline
3. friend
4. static

9. 멤버 함수 내부에서 객체의 멤버 변수를 변경하지 못하도록 읽기 전용 함수로 선언할 때, 빈칸 (A)에 들어갈 올바른 위치와 키워드는 무엇인가? (중)

```
class Player {
    int hp = 100;
public:
    int GetHp() ( A ) {
        return hp;
    }
};
```

1. 함수 이름 앞의 **const**
2. 매개변수 목록 뒤의 **const**
3. 반환 타입 앞의 **mutable**
4. 함수 본문 안쪽 시작 부분의 **constexpr**

10. 클래스의 전방 선언(**Forward Declaration**)만 수행된 상태에서 가능한 작업으로 올바른 것은? (상)

1. 전방 선언된 타입의 값을 직접 저장하는 멤버 변수 선언
2. 전방 선언된 타입의 객체를 **new** 연산자로 직접 생성
3. 전방 선언된 타입에 대한 포인터(**Type***) 또는 참조(**Type&**) 변수 선언
4. 전방 선언된 타입의 멤버 함수를 직접 호출

11. 헤더 파일의 중복 포함을 방지하기 위한 헤더 가드(**Header Guard**) 작성 시 빈칸 (A)에 들어갈 전처리 지시자는 무엇인가? (중)

```
#ifndef PLAYER_H
( A ) PLAYER_H

class Player { };

#endif
```

1. **#include**
2. **#define**
3. **#pragma**
4. **#undef**

12. 클래스 정의 내부에 멤버 함수의 선언과 구현을 함께 작성했을 때 나타나는 C++ 컴파일러의 기본 동작으로 올바른 것은? (중)

1. 컴파일 오류를 발생시킨다.
2. 암시적으로 인라인(**inline**) 함수로 처리된다.
3. 자동으로 정적(**static**) 함수로 전환된다.
4. 외부 파일에서 호출할 수 없는 **private** 함수가 된다.

13. 객체의 불변식(**Invariant**)과 캡슐화 설계에 대한 설명으로 올바른 것은? (중)

1. 모든 **private** 멤버 변수에 대해 기계적으로 **public** 게터와 세터를 작성하는 것이 캡슐화의 완성이다.
2. 세터(**Setter**) 함수는 검증 로직 없이 외부에서 전달된 값을 무조건 대입해야 한다.
3. 객체가 항상 유효한 상태를 유지하도록 상태 변경 규칙을 포함한 의미 있는 함수(예: **TakeDamage**)를 제공하는 것이 바람직하다.

4. 캡슐화를 적용하면 프로그램의 실행 속도가 항상 2배 이상 향상된다.

14. C++ 클래스 정의 문법에서 닫는 중괄호 뒤에 반드시 붙여야 하는 기호로 올바른 것은?
(하)

```
class MyClass {  
    // 멤버 선언  
}( A )
```

1. 아무것도 붙이지 않는다.
2. : (콜론)
3. ; (세미콜론)
4. , (coma)

15. 객체지향 프로그래밍의 캡슐화(Encapsulation)와 정보 은닉(Information Hiding)에 대한 설명으로 올바른 것은? (하)

1. 객체의 내부 구현 세부사항을 외부에 노출시켜 자유롭게 수정할 수 있도록 돕는다.
2. 데이터(상태)와 관련 기능(동작)을 하나로 묶고, 필요한 인터페이스만 외부에 공개하여 내부 상태를 보호한다.
3. 프로그램의 모든 변수를 전역 변수화하는 기법이다.
4. 상속 계층 구조를 복잡하게 만들어 코드를 재사용하는 기법이다.

16. C++의 friend 키워드 성격에 대한 설명으로 옳지 않은 것은? (상)

1. friend 선언은 단방향 관계이다. (A가 B를 friend로 지정해도 B가 A의 private 멤버에 접근할 수 없다.)
2. friend 관계는 상속되지 않는다. (기반 클래스의 friend가 파생 클래스의 private 멤버까지 자동으로 접근할 수 없다.)
3. friend 관계는 전이되지 않는다. (A가 B를, B가 C를 friend로 지정해도 C가 A의 private에 접근할 수 없다.)
4. friend 함수로 지정되면 해당 함수는 자동으로 클래스의 멤버 함수가 되어 this 포인터를 갖는다.

17. 다음 코드에서 ClassB가 ClassA의 포인터만 멤버로 가지고 있을 때, ClassA.h를 #include하는 대신 전방 선언을 활용하는 올바른 코드 빈칸 (A)는 무엇인가? (중)

```
// ClassB.h  
( A )  
  
class ClassB {  
    ClassA* ptrA;  
};  
  
class ClassA;
```

```
include ClassA;
using ClassA;
typedef ClassA;
```

18. UML 클래스 다이어그램에서 전체 객체가 소멸할 때 부분 객체도 함께 소멸하며, 독점적인 포함 관계를 나타내는 속이 꺾 찬 마름모 기호의 관계는 무엇인가? (중)

1. 일반화(Generalization)
2. 합성(Composition)
3. 집합(Aggregation)
4. 의존(Dependency)

19. UML 클래스 다이어그램에서 한 클래스가 다른 클래스를 매개변수, 지역 변수 또는 반환 타입 등으로 일시적으로 사용하여 점선 화살표로 표현되는 관계는 무엇인가? (중)

1. 의존(Dependency)
2. 합성(Composition)
3. 일반화(Generalization)
4. 연관(Association)

20. 클래스 외부에서 public 정적 멤버 변수 maxScore에 접근하여 값을 읽거나 변경할 때 사용하는 올바른 문법은 무엇인가? (단, 클래스 이름은 Game이다.) (하)

1. Game->maxScore
2. Game.maxScore
3. Game::maxScore
4. Game[maxScore]

21. 다음 코드에서 객체의 불변식(HP는 0 미만이 될 수 없음)을 지키기 위한 세터 함수의 빈칸 (A)에 들어갈 올바른 조건식은 무엇인가? (중)

```
class Player {
    int hp = 100;
public:
    void SetHp(int value) {
        if (( A )) {
            hp = 0;
        } else {
            hp = value;
        }
    }
};
```

1. value > 100
2. value < 0

3. value == 0
4. value != 100

22. 클래스(Class)와 객체(Object / Instance)의 관계에 대한 설명으로 올바른 것은? (하)

1. 클래스는 메모리에 실제 할당된 실체이고, 객체는 타입의 정의이다.
2. 클래스는 객체를 만들어내기 위한 틀(타입)이며, 객체는 클래스를 바탕으로 메모리에 실재하게 된 인스턴스이다.
3. 하나의 클래스로는 단 하나의 객체만 생성할 수 있다.
4. 객체들이 공유하는 메모리 공간을 클래스라고 부른다.

23. 다음 코드처럼 객체의 내부 문자열 데이터를 복사 비용 없이 읽기 전용으로 안전하게 반환하기 위해 빈칸 (A)에 들어갈 올바른 반환 타입은 무엇인가? (상)

```
class User {  
    std::string name;  
public:  
    ( A ) GetName() const {  
        return name;  
    }  
};
```

1. std::string
2. const std::string&
3. std::string*
4. void

24. UML 시퀀스 다이어그램에서 특정 객체가 생존해 있는 기간을 아래 방향의 세로 점선으로 표현하는 요소는 무엇인가? (하)

1. 생명선(Lifeline)
2. 실행 구간(Activation)
3. 반환 메시지(Return Message)
4. 제어 블록(Control Block)

25. 다음 코드에서 Printer 클래스가 Document 클래스의 private 데이터 멤버 content를 직접 읽을 수 있도록 허용하는 빈칸 (A)의 선언은 무엇인가? (상)

```
class Document {  
private:  
    std::string content = "Secret Data";  
    ( A );  
};  
  
class Printer {  
public:
```

```
void Print(const Document& doc) {  
    std::cout << doc.content;  
}  
};
```

1. friend class Printer
2. public class Printer
3. protected Printer
4. virtual class Printer